

ADAM : A Method for Stochastic Optimization

날짜	2026년 5월 11일
태그	Optimization

Paper Review

Adam: A Method for Stochastic Optimization

We introduce Adam, an algorithm for first-order gradient-based optimization of stochastic objective functions, based on adaptive estimates of lower-order moments. The method is straightforward to...

<https://arxiv.org/abs/1412.6980>



Keyword : Optimization, Stochastic, Adaptive Learning Rate

1. Introduction

논문이 다루는 분야

Stochastic Optimization for Machine Learning

- 대규모 데이터셋과 고차원 파라미터 공간을 가진 머신러닝 문제에서 사용 가능한 **stochastic objective function 최적화 방법**에 대해 다룸
- Noisy/sparse gradient, non-stationary objective 등 다양한 stochastic 환경에서 효율적으로 작동하는 것이 목표

해당 task에서 기존 연구 한계점

- **기존 최적화 방법 : Stochastic gradient-based optimization**
 - 과학 및 공학 분야에서 스칼라 파라미터화된 **목적 함수의 최적화**(목적 함수 최대화/최소화)가 핵심
 - 이때 목적 함수가 미분 가능하면 **gradient descent**는 상대적으로 효율적인 최적화 방법 (모든 파라미터에 대한 1차 편미분 계산이 함수를 평가하는 것과 계산 복잡도가 동일함)
 - 많은 목적 함수가 데이터 서브샘플에서 평가되는 서브함수들의 합으로 구성되어 있어 각 서브함수에 대해 gradient step을 취하는 **stochastic gradient descent**로 효율적으로 최적화 가능
 - SGD는 많은 성공적인 기계 학습 연구의 중심에 있는 효율적이고 효과적인 최적화 기법
- **논문에서 다루는 문제**
 - 데이터 서브샘플링 외에도 dropout 같은 정규화 기법 등 다른 노이즈 원인이 존재
→ 이러한 모든 **noisy objective**에 대해 효율적인 stochastic 최적화 기법이 필요
 - **고차원 파라미터** 공간을 가진 stochastic objective의 최적화에 초점
→ 고차원에서는 고차 최적화 방법이 부적합하므로 1차 방법(1차 미분)에 한정하여 논의

논문의 contributions

- **Contributions : Adam**
 - 1차 gradient만으로 효율적인 stochastic 최적화를 수행하는 방법
 - Gradient의 1차 및 2차 모멘트 추정치로부터 각 파라미터에 대해 개별적인 adaptive 학습률을 계산
→ **Adaptive moment estimation**
 - 두 방법의 장점을 결합하여 설계
 - **AdaGrad** : sparse gradient 처리에 효과적

- **RMSProp** : online 및 non-stationary 환경에서 효과적
- **장점**
 - Parameter update의 크기가 gradient의 rescaling에 불변
 - Step size가 stepsize hyperparameter에 의해 대략적으로 제한됨
 - Stationary 목적 함수를 요구하지 않음
 - Sparse gradient에서 작동
 - Step size annealing을 자연스럽게 수행

2. Related Work

- Adam에 직접적으로 관련된 최적화 방법은 RMSProp과 AdaGrad
- **기타 관련 방법들**
 - **vSGD, AdaDelta, natural Newton method** : 1차 정보로부터 곡률을 추정하여 stepsize를 설정
 - **Sum-of-Functions Optimizer(SFO)** : minibatch 기반 quasi-Newton 방법이지만, 메모리 요구량이 minibatch 파티션 수에 비례하여 GPU 등 메모리가 제한적인 환경에서 실현하기 어려움
 - **Natural Gradient Descent (NGD)** : NGD처럼 Adam도 데이터의 기하학적 구조에 맞추는 preconditioner를 사용하지만, Adam의 preconditioner는 vanilla NGD보다 보수적

RMSProp

- **RMSProp with momentum과 Adam의 차이점**
 1. **Parameter update 방식**
 - a. **RMSProp** : rescaled gradient에 momentum을 적용하여 parameter update
 - b. **Adam** : gradient의 1차 및 2차 모멘트의 running average로 직접 update를 추정
 2. **Bias-correction term**

RMSProp은 Adam과 달리 bias-correction term이 없음
→ β_2 가 1에 가까운 경우(sparse gradient에서 필요) 매우 큰 stepsize로 이어져 발산 가능

AdaGrad

- Sparse gradient에 효과적인 알고리즘
- **Basic 버전의 파라미터 업데이트 식** : $\theta_{t+1} = \theta_t - \alpha \cdot g_t / \sqrt{\sum_{i=1}^t g_i^2}$
- Adam에서 $\beta_1 = 0$, $(1 - \beta_2)$ 를 극소, $\alpha_t = \alpha \cdot t^{-1/2}$ 로 설정하면 AdaGrad와 일치
 - Bias correction이 없으면 이런 관계가 성립하지 않음(무한히 큰 bias와 parameter update 발생)

3. 제안 방법론

Main Idea

Algorithm

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)

- 목적 : noisy 목적 함수 $f(\theta)$ 의 기댓값 $\mathbb{E}[f(\theta)]$ 를 최소화

- $f_1(\theta), \dots, f_T(\theta)$: timestep 1, ..., T 에서의 stochastic 함수값
- $g_t = \nabla_{\theta} f_t(\theta)$: timestep t 에서의 gradient

• Parameter Update

- Gradient의 exponential moving average(m_t)와 Squared gradient의 exponential moving average(v_t)를 업데이트

- $\beta_1, \beta_2 \in [0, 1)$: moving average의 exponential decay rate를 제어하는 하이퍼파라미터
- m_t : 1차 모멘트(mean)의 추정치
- v_t : 2차 모멘트(uncentered variance)의 추정치

• Initialization Bias

- $m_0 = 0, v_0 = 0$ 으로 초기화 → 학습 초기에 모멘트 추정치가 0에 편향됨. 특히, decay rate가 크면 (β 가 1에 가까울수록) 편향이 심해짐.
- Bias-corrected estimate $\hat{m}_t, \hat{v}_t$ 로 보정 필요
- 마지막 세 줄을 $\alpha_t = \alpha \cdot \sqrt{1 - \beta_2^t} / (1 - \beta_1^t)$ 와 $\theta_t \leftarrow \theta_{t-1} - \alpha_t \cdot m_t / (\sqrt{v_t} + \epsilon)$ 로 대체해서 계산 효율화 가능

Adam's Update Rule

- **Stepsize** : Adam에서 가장 중요한 것은 stepsize를 잘 선택하는 것
 - **Effective stepsize** (at timestep t) : $\Delta_t = \alpha \cdot \hat{m}_t / \sqrt{\hat{v}_t}$ ($\epsilon = 0$ 가정)
 - **Upper bound**
 1. Severely sparse case - 현재 timestep을 제외한 모든 timestep에서 gradient가 0인 경우
: $(1 - \beta_1) > \sqrt{1 - \beta_2}$ 이면, $|\Delta_t| \leq \alpha \cdot (1 - \beta_1) / \sqrt{1 - \beta_2}$

2. Less sparse case - 일반적인 경우

: $|\Delta_t| \leq \alpha \rightarrow \text{smaller}$

일반적인 경우 $|\Delta_t| \leq \alpha$ 인 이유

$\hat{m}_t$ 는 gradient의 가중 평균, $\hat{v}_t$ 는 gradient 제곱의 가중 평균이므로, $\hat{m}_t/\sqrt{\hat{v}_t}$ 는 평균 / $\sqrt{(\text{제곱의 평균})}$ 의 구조를 가짐.

통계적으로 $|E[g]|/\sqrt{(E[g^2])} \leq 1$ 이 항상 성립하므로, $|\hat{m}_t/\sqrt{\hat{v}_t}| \leq 1$ 가 되어 $|\Delta_t| = \alpha \cdot |\hat{m}_t/\sqrt{\hat{v}_t}| \leq \alpha$ 특히, $(1 - \beta_1) = \sqrt{(1 - \beta_2)}$ 이면 $|\hat{m}_t/\sqrt{\hat{v}_t}| < 1$ 이 보장되어 $|\Delta_t| < \alpha$

$\rightarrow |\Delta_t| \lesssim \alpha$: stepsize setting α 에 의해 대략적으로 이동 범위의 크기가 결정됨

• Trust region α

- 현재 파라미터 값 주변에 trust region을 설정하는 것과 같음 $\rightarrow$ 이 영역을 벗어나면 현재 gradient 추정치가 충분한 정보를 제공하지 못함
- α 의 적절한 크기를 사전에 파악하기 비교적 쉬움

• Signal-to-Noise Ratio (SNR) : $\hat{m}_t/\sqrt{\hat{v}_t}$

- SNR이 작을수록, Δ_t 가 0에 가까워짐
 - SNR이 작다는 것은 $\hat{m}_t$ 의 방향이 실제 gradient 방향과 일치하는지에 대한 불확실성이 크다는 의미
- Optimum에 가까워질수록 SNR이 0에 수렴 $\rightarrow$ 자동 annealing 효과

• Scale invariance

- Gradient g 를 factor c 로 rescaling해도 effective stepsize가 변하지 않음
- $\hat{m}_t$ 는 c 배, $\hat{v}_t$ 는 c^2 배가 되어 $(c \cdot \hat{m}_t)/\sqrt{(c^2 \cdot \hat{v}_t)} = \hat{m}_t/\sqrt{\hat{v}_t}$ 로 상쇄

Initialization Bias Correction

- **Initialization Bias 문제** : $v_0 = 0$ 으로 초기화 시, v_t 가 실제 $\mathbb{E}[g_t^2]$ 보다 작게 편향됨 (여기서는 2차 모멘트를 다루지만, 1차 모멘트도 유사하게 실제 값과 차이가 발생)
- **Correction**

v_t update 식 $v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ 을 이전 모든 timestep의 gradient의 함수로 나타내면,

$$v_t = (1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} \cdot g_i^2$$

이때 $\mathbb{E}[v_t]$ 가 실제 2차 모멘트 $\mathbb{E}[g_t^2]$ 와 얼마나 차이가 있는지를 알아보기 위해 양변에 기댓값을 적용하면,

$$\begin{aligned} \mathbb{E}[v_t] &= \mathbb{E} \left[(1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} \cdot g_i^2 \right] \\ &= \mathbb{E}[g_t^2] \cdot (1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} + \zeta \\ &= \mathbb{E}[g_t^2] \cdot (1 - \beta_2^t) + \zeta \end{aligned}$$

- $\zeta = 0$: 실제 2차 모멘트 $\mathbb{E}[g_t^2]$ 가 stationary인 경우
- $\zeta \neq 0$ 이어도 β_2 를 적절히 선택하면, 오래된 gradient에 작은 가중치를 부여하여 ζ 를 작게 유지 가능

ζ 를 무시하면, $\mathbb{E}[v_t]$ 와 $\mathbb{E}[g_t^2]$ 사이에 $(1 - \beta_2^t)$ 항이 남는데, 이것이 **Initialization Bias** 이다. 따라서 v_t 를 $(1 - \beta_2^t)$ 으로 나누어 보정한다.

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

• Sparse gradient에서의 중요성

Sparse gradient에서 신뢰할 수 있는 2차 모멘트 추정을 위해서는 작은 β_2 를 선택해 많은 gradient를 평균내야 한다. 그러나 β_2 가 작을수록 $(1 - \beta_2^t)$ 가 빠르게 1에 수렴하지 않기 때문에 **bias correction이 없으면 초기 step이 과도하게 커지는 문제**가 발생

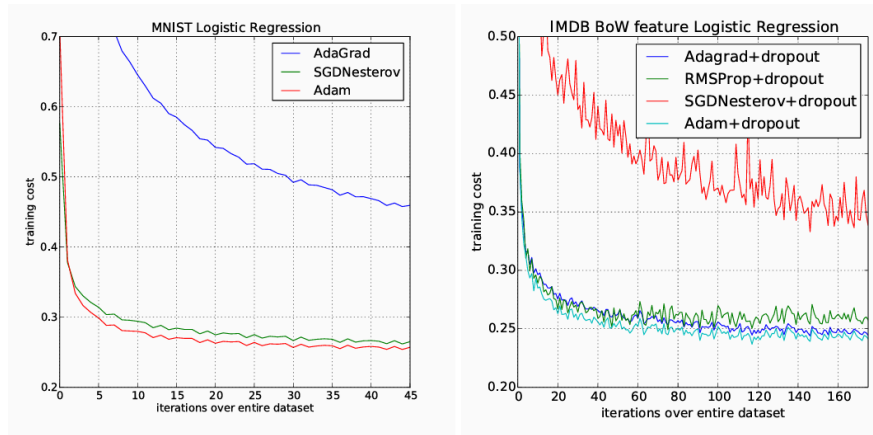
4. 실험 및 결과

Baseline & 결과

다양한 머신러닝 모델(logistic regression, multilayer fully connected neural networks, Deep CNN 등)에서 Adam을 실험적으로 평가

- 최적화 알고리즘 비교 시 동일한 파라미터 초기화 사용
- Learning rate, momentum 등 하이퍼파라미터는 dense grid 탐색 후, 최적 설정으로 기록

1) Experiment: Logistic Regression



Logistic regression은 well-studied convex objective를 가지므로, local minimum 문제 없이 서로 다른 optimizer를 순수하게 비교할 수 있음.

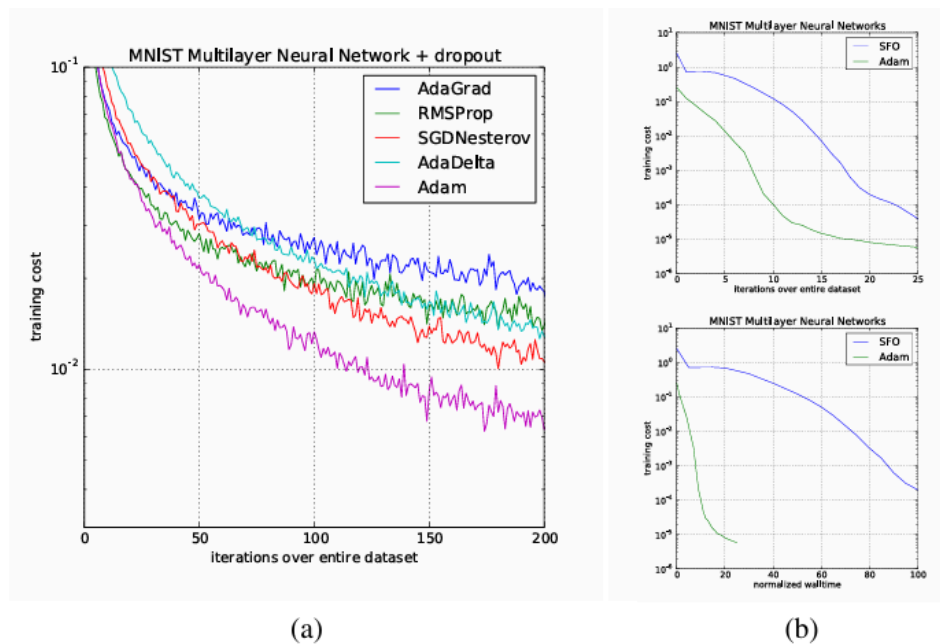
MNIST)

- **설정** : MNIST 데이터셋에서 L2-regularized multi-class logistic regression으로 평가
 - Stepsize에 $1/\sqrt{t}$ decay 적용
 - 비교 대상 : SGD with Nesterov momentum, AdaGrad (minibatch size 128)
- **결과**
 - Adam이 SGD with momentum과 유사한 수렴 속도를 보였으며, 둘 다 AdaGrad보다 빠르게 수렴

IMDB movie review dataset) Sparse feature problem

- **설정** : 10,000차원의 highly sparse feature vector 사용 + 50% dropout
- **결과**
 - AdaGrad가 SGD with Nesterov momentum보다 큰 차이로 우수
 - Adam은 AdaGrad만큼 빠르게 수렴
 - Adam이 sparse feature의 장점을 잘 활용하여 SGD with momentum보다 빠른 수렴 가능

2) Experiment: Multi-layer Neural Networks

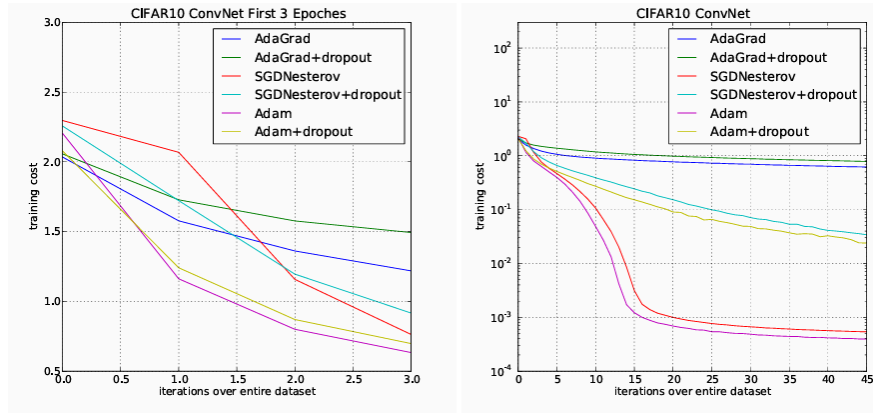


Non-convex 목적 함수를 가진 multi-layer neural network에서 평가. 수렴 분석이 non-convex에는 적용되지 않지만, 실험적으로 Adam이 다른 방법들을 자주 능가함을 확인.

- **설정** : MNIST 데이터셋, 2개의 hidden layer (각 1000 units, ReLU), minibatch size 128
- **결과**
 1. **Deterministic cross-entropy objective function 비교**
 - Adam이 iteration 수와 wall-clock time 모두에서 SFO보다 더 빠른 progress
 - SFO는 곡물 정보 업데이트 비용으로 iteration당 5-10배 느리며, 메모리도 minibatch 수에 비례
 2. **Stochastic regularization method - dropout**

- SFO는 deterministic subfunctions를 가정하므로, stochastic regularization이 있는 cost function에서 수렴 실패
- Adam이 다른 stochastic 1차 방법들(AdaGrad, RMSProp, SGD Nesterov, AdaDelta)보다 더 나은 수렴도를 보임

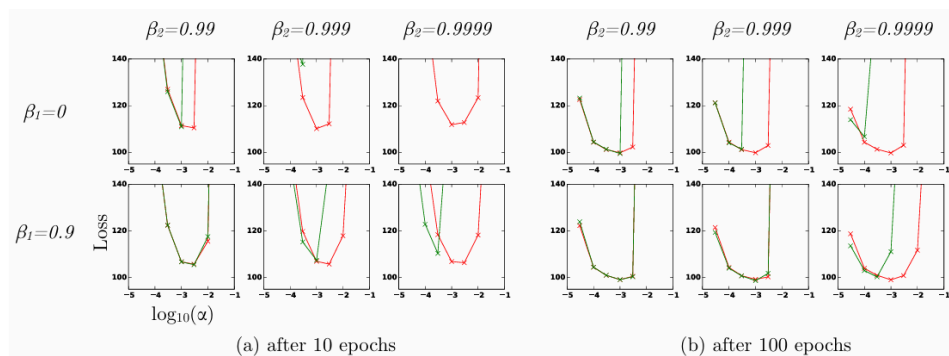
3) Experiment: Convolutional Neural Networks



CNN은 weight sharing으로 인해 레이어별로 gradient 크기가 크게 달라, SGD 적용 시 보통 작은 학습률을 사용함.

- **설정** : CIFAR-10 데이터셋, 3단계 5×5 conv + 3×3 max pooling (stride 2) + FC 1000 (ReLU), dropout, minibatch 128
- **결과**
 - **빠른 수렴 속도** : 초기 학습 단계에서 Adam과 AdaGrad 모두 빠르게 training cost를 낮췄으나, 후반에는 Adam과 SGD가 AdaGrad보다 훨씬 빠르게 수렴
 - **Adam vs SGD** : Adam이 SGD with momentum 대비 소폭 개선됐지만, SGD처럼 레이어별 학습률을 수동 설정할 필요 없이 자동 적응한다는 장점이 있음
 - **2차 모멘트** : CNN에서는 $\hat{v}_t$ 가 몇 epoch 이후에 0으로 수렴하여 ϵ 에 지배됨 → FC network에 비해 CNN의 cost function 구조에서는 2차 모멘트 근사가 좋지 않음
 - **1차 모멘트** : CNN에서는 1차 모멘트를 통한 minibatch 분산 감소가 속도 향상에 더 중요한 역할

4) Experiment: Bias-Correction Term



Bias correction의 효과를 실험적으로 검증. Bias correction을 제거하면 RMSProp with momentum과 동일해짐.

- **설정** : VAE (1 hidden layer, 500 units, softplus, 50차원 Gaussian latent variable)
 - $\beta_1 \in [0, 0.9], \beta_2 \in [0.99, 0.999, 0.9999], \log_{10}(\alpha) \in [-5, \dots, -1]$

- **결과**

- β_2 가 1에 가까울 때, 큰 initialization bias가 발생하여 초반 몇 epoch에 불안정함 → Slow decay에서 bias correction이 중요
- 작은 $(1 - \beta_2)$ 와 bias correction의 조합에서 최상의 결과
- 학습 후반에 gradient가 sparser해질수록 (hidden unit이 특정 패턴에 특화됨에 따라) bias correction의 효과가 더 두드러짐

5. 결론

- **Contributions**

- Stochastic objective function의 gradient 기반 최적화를 위한 알고리즘 **Adam**을 제안
- AdaGrad의 sparse gradient 처리 능력과 RMSProp의 non-stationary objective 처리 능력을 결합한 알고리즘
- **장점**
 - 대규모 데이터셋 및 고차원 파라미터 공간의 머신러닝 문제에 적합
 - 구현이 간단하고 메모리 요구량이 적음

- **실험 결과**

- Convex 및 non-convex 최적화 문제에서도 robust하고 우수한 성능을 보임

Efficient Memory Management for Large Language Model Serving with PagedAttention

날짜 2026년 5월 11일

태그 Serving

Paper Review

Efficient Memory Management for Large Language Model Serving with...

High throughput serving of large language models (LLMs) requires batching sufficiently many requests at a time. However, existing systems struggle because the key-value cache (KV cache) memory for...

<https://arxiv.org/abs/2309.06180>



Keyword : LLM Serving, KV Cache Memory Management, Virtual Memory

1. Introduction

논문이 다루는 분야

High-throughput LLM Serving

- GPT, PaLM 등 대규모 언어 모델을 프로그래밍 어시스턴트, 범용 챗봇 등 다양한 서비스에 활용할 때 중요시되는 **serving 효율**에 대해 다룸
- LLM serving의 핵심 과제는 throughput을 높여 요청당 비용을 줄이는 것
- 충분히 많은 요청을 batching해야 하지만, 각 요청의 KV cache 메모리가 크고 동적으로 변하여 효율적인 메모리 관리가 필요

해당 task에서 기존 연구 한계점

- **LLM serving의 memory-bound 문제**
 - LLM의 핵심인 autoregressive Transformer는 토큰을 하나씩 순차 생성하는 구조로, workload가 memory-bound가 되어 GPU 연산 자원을 충분히 활용하지 못함
 - Throughput 향상을 위해 여러 요청을 batching할 수 있지만, 이를 위해서는 각 요청의 메모리를 효율적으로 관리해야 함
 - **GPU 메모리 분포** (13B 모델, A100 40GB 기준) : 모델 가중치 약 65% (고정), KV cache 약 30% (동적), 활성화 등 기타 소량 → 모델 가중치는 고정이고 활성화는 소량이므로, **KV cache 관리 방식**이 최대 batch size를 결정하는 핵심 요인
- **KV cache 메모리 관리의 비효율성**
 - 기존 시스템은 KV cache를 연속적인 메모리 공간에 저장하는데, KV cache는 토큰 생성에 따라 동적으로 크기가 변하고 생성 전에 길이를 알 수 없어 비효율이 발생
- **Internal & External fragmentation**
 - **내부 단편화** : 요청의 최대 길이(예: 2048 토큰)만큼 연속 메모리를 미리 할당하므로, 실제 길이가 짧으면 내부 단편화가 발생
 - **외부 단편화** : 요청마다 할당 크기가 달라 외부 단편화도 발생
 - 기존 시스템에서 실제 토큰 저장에 사용되는 KV cache 메모리는 전체의 20.4%-38.2%에 불과
- **메모리 공유 불가**
 - Beam search 등 하나의 요청에서 여러 출력 시퀀스를 생성하는 경우, 시퀀스 간 KV cache를 부분적으로 공유할 수 있음

- 그러나 각 시퀀스의 KV cache가 별도의 연속 메모리 공간에 저장되므로 공유가 불가능

논문의 contributions

- **PagedAttention** 제안 : OS의 virtual memory와 paging에서 영감을 받은 attention 알고리즘
 - KV cache를 고정 크기의 block으로 나누어 비연속적인 메모리에 저장
 - Block = page, token = byte, request = process로 대응하여 유연한 메모리 관리
 - 작은 block 단위의 on-demand 할당으로 내부 단편화 완화, 균일한 block 크기로 외부 단편화 제거
 - Block 단위의 메모리 공유 가능 (같은 요청의 여러 시퀀스 간, 또는 서로 다른 요청 간)
- **vLLM** 구축 : PagedAttention 위에 구축된 고처리량 분산 LLM serving 엔진
 - Block-level 메모리 관리와 preemptive request scheduling을 PagedAttention과 공동 설계
 - GPT, OPT, LLaMA 등 다양한 크기의 LLM을 지원 (단일 GPU 메모리 초과 모델 포함)
- **성과**
 - LLM serving에서의 메모리 할당 문제를 식별하고 serving 성능에 미치는 영향을 정량화
 - 기존 SOTA 시스템(FasterTransformer, Orca) 대비 2-4배 throughput 향상, 모델 정확도 손실 없음
 - 긴 시퀀스, 큰 모델, 복잡한 디코딩 알고리즘에서 개선 효과가 더 두드러짐